

🔥 Perfect. Now we move to **REAL ENGINE BUILDING**.

Your app is no longer a UI shell — now we turn it into a **true fitness system**.

PHASE 2: CORE WORKOUT ENGINE (MOST IMPORTANT PHASE)

👉 This is where your app becomes **better than most fitness apps**

PHASE 2 OBJECTIVE

Convert your app from:

❌ Fixed small workouts



✅ **Dynamic, intelligent workout system with 1000+ exercises**

WHAT YOU WILL BUILD

After this phase:

- ✅ 1000+ exercise dataset
 - ✅ Smart workout generator
 - ✅ Different workouts every time
 - ✅ Structured plans (warmup → main → finisher)
 - ✅ Real variety (not repetitive)
-

PHASE 2 STRUCTURE

We break into 3 sprints:

🎯 SPRINT 2.1 — EXERCISE DATASET SYSTEM

🎯 Goal: Build **foundation** (1000+ exercises)

🎯 SPRINT 2.2 — WORKOUT GENERATOR ENGINE

🎯 Goal: Create **smart workout builder**

🎯 SPRINT 2.3 — WORKOUT PLAYER (execution)**

🎯 Goal: Make workouts **actually usable**

🔧 FINAL REPLIT PROMPT (PHASE 2 COMPLETE)

Paste this EXACTLY 📌

You are a senior fitness app architect and React Native engineer.

You must COMPLETE PHASE 2 of the FitAI app:

Build a REAL workout engine with large dataset + smart generator.

⚠️ DO NOT use small static exercise lists.

⚠️ DO NOT hardcode workouts.

🎯 OBJECTIVES

1. Create 1000+ exercise dataset
2. Categorize exercises properly
3. Build dynamic workout generator
4. Ensure workouts are different every time
5. Support home + gym + levels

STEP 1 — CREATE EXERCISE DATASET

Create:

/data/exercises.json

Structure:

```
[
{
  "id": "ex_001",
  "name": "Push-Up",
  "muscle": "chest",
  "secondaryMuscles": ["triceps", "shoulders"],
  "equipment": "bodyweight",
  "level": "beginner",
  "type": "strength",
  "duration": 30,
  "reps": 12,
  "calories": 5,
  "image": "pushup.gif"
}
```

REQUIREMENTS

Dataset must include:

- Chest (100+)
- Abs (100+)
- Legs (100+)
- Arms (100+)
- Back (100+)
- Shoulders (100+)
- Full Body (100+)

 Total: 700–1000+ exercises



STEP 2 — UTILITY FUNCTIONS

Create:

/utils/exerciseUtils.js

Functions:

1. getExercisesByMuscle(muscle)
 2. filterByLevel(level)
 3. filterByEquipment(type)
 4. shuffleExercises(array)
-



STEP 3 — WORKOUT GENERATOR

Create:

/utils/workoutGenerator.js

Main function:

```
generateWorkout({  
  muscle,  
  level,  
  duration,  
  location  
})
```



LOGIC

BEGINNER:

- 5–6 exercises
- simple movements

INTERMEDIATE:

- 6–8 exercises
- mix compound + isolation

ADVANCED:

- 8–12 exercises
 - include explosive + supersets
-

DURATION LOGIC

Short (<10 min):

- 4–5 exercises

Medium (10–25 min):

- 6–8 exercises

Long (25+ min):

- 8–12 exercises
-



LOCATION LOGIC

HOME:

- bodyweight only

GYM:

- include machines + weights
-



STEP 4 — WORKOUT STRUCTURE

Workout must follow:

1. Warm-up (1–2 exercises)
 2. Main workout (core exercises)
 3. Finisher (high intensity)
-



STEP 5 — RANDOMIZATION

Ensure:

- Shuffle exercises
 - No repetition
 - Different result every time
-



STEP 6 — INTEGRATE INTO APP

When user clicks:

Start Workout

Replace static list with:

```
const workout = generateWorkout(...)
```



STEP 7 — DISPLAY DATA

Each exercise card must show:

- Name
 - Reps or duration
 - Calories
 - Muscle tag
 - Image/GIF
-



STEP 8 — VALIDATION

Test:

1. Generate chest workout → different every time
 2. Beginner vs Advanced → different intensity
 3. Home vs Gym → different exercises
 4. No duplicates in same workout
-



FINAL RESULT

After implementation:

- ✓ Each workout is unique
 - ✓ Large variety of exercises
 - ✓ Real structured plans
 - ✓ Scalable dataset system
-



RULE

Do NOT:

- Hardcode workouts
- Use small arrays
- Repeat same exercises

Build a real engine.

+ animations + AI interaction